

INF1103: Lab 1

Foundations of Programming and Dev Ops

1. Introduction

Welcome to your first lab session for INF1103! In this lab, we will calibrate and establish the essential workspace toolkit needed to kickstart your journey into modern software engineering. This session acts as the absolute baseline for all subsequent lab activities, so it is highly important that you not only complete it but thoroughly understand every system metaphor presented here.

Remember, the focus of this module is all about bridging logical programming constructs with modern, automated operational practices. We will systematically evolve our applications week by week, refactoring and packaging our code to fit enterprise-grade deployment standards.

True engineering mastery comes from hands-on problem-solving. Ensure you work through and complete each step of this lab by yourself. While your peers are excellent resources for discussing conceptual architectures, they must not write code for you or share their files with you!

It is highly critical that you have watched and engaged with the Week 1 lecture lessons before attempting this lab. Do not hesitate to ask plenty of questions to your professor and teaching assistants when you hit an infrastructure error. We are here to support you. Good luck!

2. Objectives

In this lab, you will set up Visual Studio Code and write your first python programs. After every feature enhancement, you will commit your work using Git to simulate an industry workflow.

1. Set Up a Development Environment

- Install and configure Visual Studio Code, Python, and Git.
- Verify that all development tools are functioning correctly.

2. Understand Version Control Fundamentals

- Initialize and configure a Git repository.
- Track file changes using Git status and diff commands.
- Stage and commit code changes using meaningful commit messages.

3. Create and Execute Python Programs

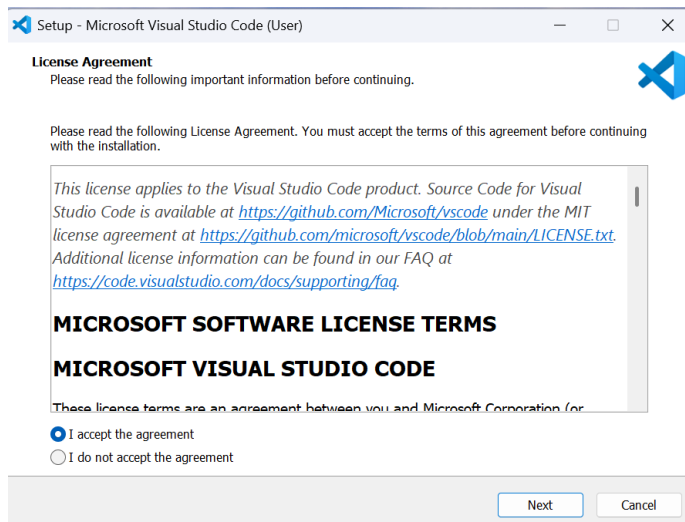
- Write, save, and run Python scripts using Visual Studio Code.
- Understand the program execution workflow.

3. Setting Up a Development Environment

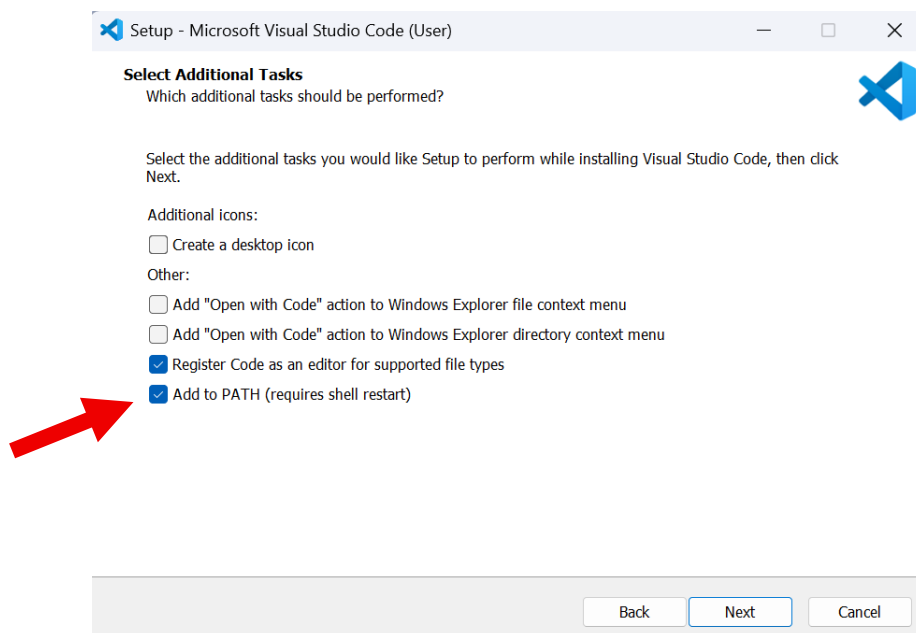
1. Install Visual Studio Code (VS Code)

VS Code is a lightweight, powerful code editor used by professional software engineers worldwide.

- a. Go to the official download page: code.visualstudio.com.
- b. Download the installer matching your operating system (Windows, macOS, or Linux).
- c. Run the installer and accept all default options.



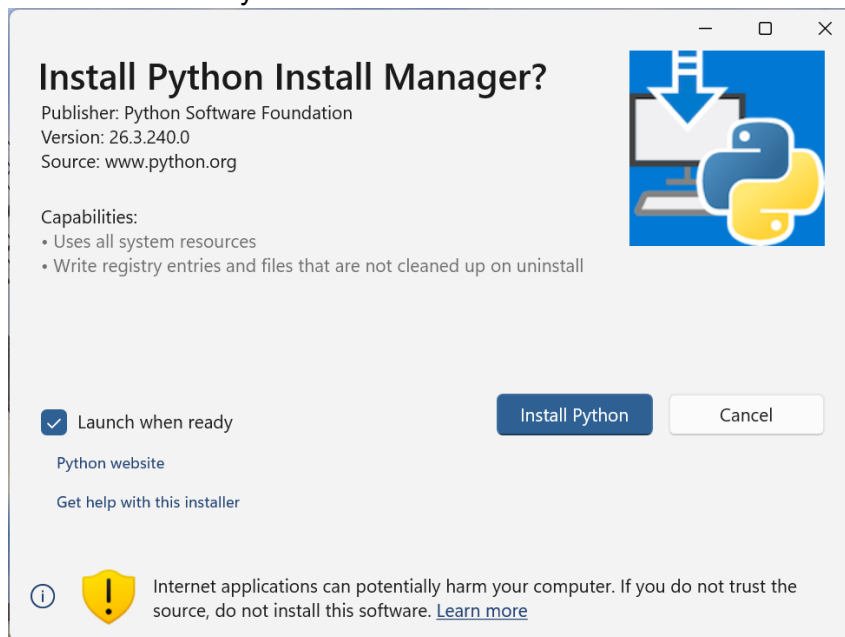
- Windows Users: Ensure you check the box that says “Add to PATH (requires shell restart)” during setup.



- Then Next to Install

2. Install Python

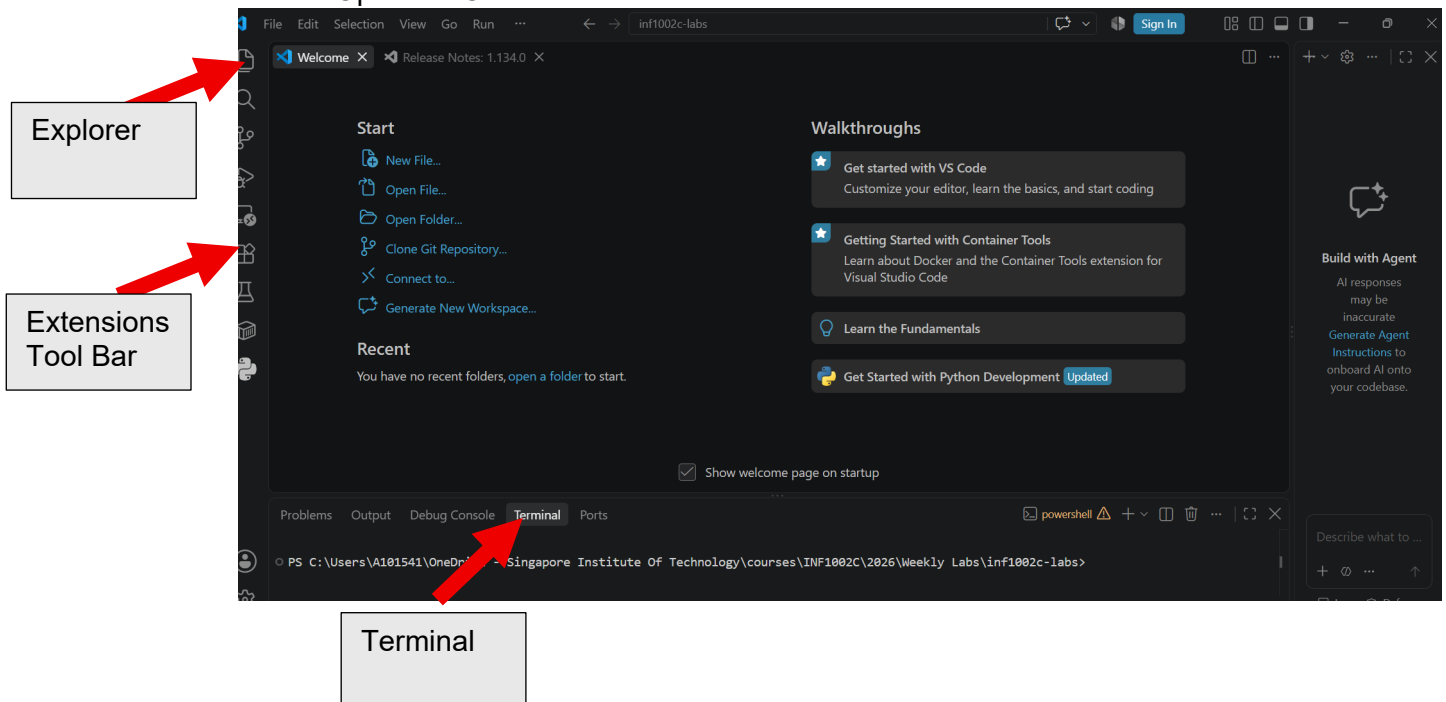
- Go to official page: <https://www.python.org/downloads/>
- Download Python Install Manager
- Install the latest Python version.



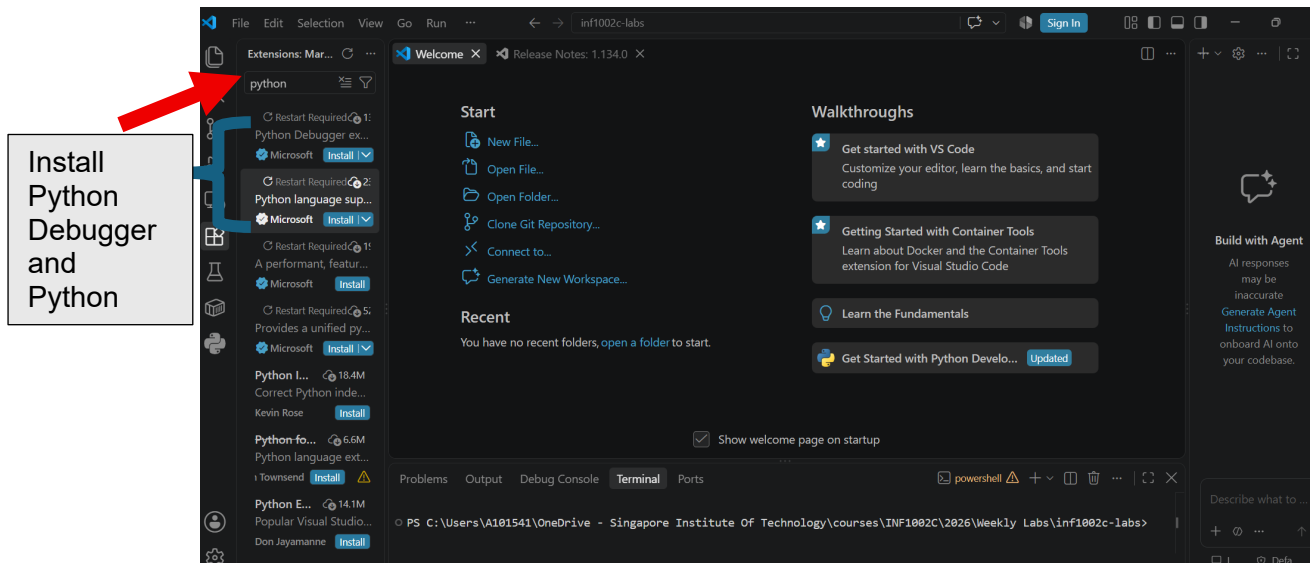
- Windows users should enable "Add Python to PATH" during installation.

3. Link Python to VS Code

- Open VS Code

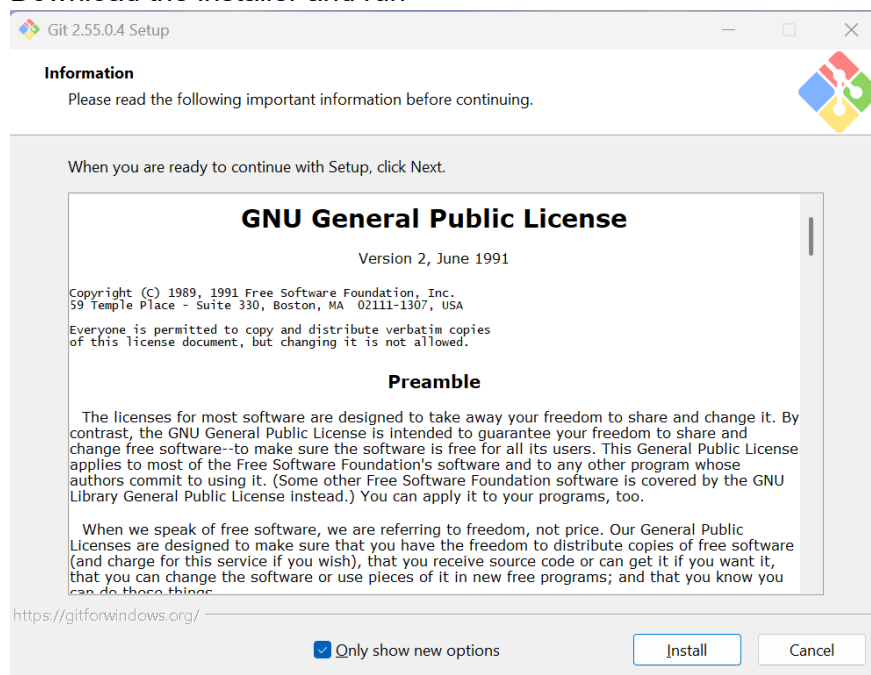


- b. Go to File-> Preferences-> Extensions or the Extension Toolbar. Type python in the Search Bar



4. Install Git

- a. Go to the official Git website: <https://git-scm.com/install/>
- b. Download the installer and run



5. Check Python and Git versions

- a. Open VSCode
- b. In Terminal:

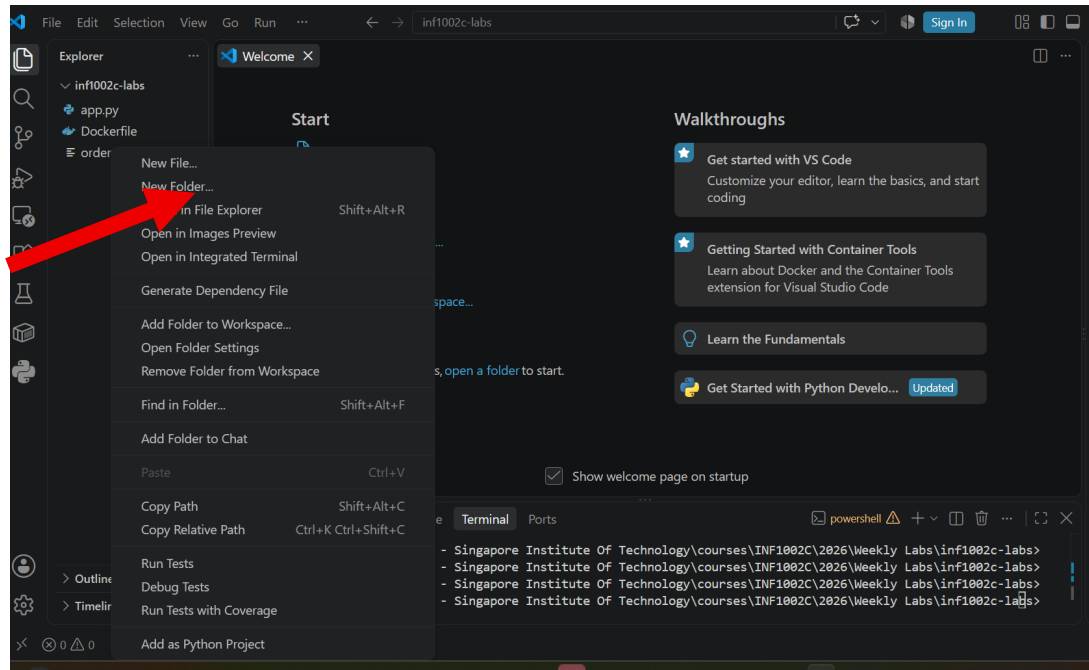
```
1 python --version
2 pip --version
3 git --version
```

- c. The output will be the respective versions

4. Understanding Version Control

Create your Development Workspace:

- Click Explorer
- Right click and choose New folder.



- Create INF1103-Labs folder in VS Code.
- Navigate to your project folder `cd INF1103-Labs`
- Configure Git for the First Time

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your_email@example.com"
```

- Initialize your git repository

```
git init
```

You will get an output message saying:
Initialized empty Git repository in yourFolder/.git/

- You can check the repo status using:

```
git status
```

The expected output is:
On branch master
No commits yet
nothing to commit

5. Python Programming

Activity 1: Create your First Python Program and add it to the repository

1. Right Click on INF1103-labs folder and click New File hello.py
2. Write your first python lines. A welcome message:

```
print("=====")
print("Welcome here")
print("My first post!")
print("=====")
```

3. Go to Run Tab and run your code, what do you observe?
 - a. Does the program display messages exactly as written? **Yes.**
 - b. What is the order of execution of the statements? top to bottom or bottom to top? **Top to bottom.**
 - c. Where does the output appear? **In the terminal.**
4. Add hello.py to your repository, In the VS Code terminal type:

```
git add hello.py
```

5. Check the status of your repo (1.g), the expected output:

```
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   hello.py
```

6. Create your first commit:

```
103-labs> git commit -m "First Post"
[master (root-commit) 14584ed] First Post
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 hello.py
```

7. View the git history by:

```
git log
```

Activity 2: Update your Profile and add it to the repository

- Let's assume you are creating an Instagram Profile and the first program was a welcome message on your page. Make the following changes to your file:

```
print("=====")
print("Welcome here")
print("My first post!")
print("=====")

username = "cool_creator"
bio = "Fun Blogger"
followers = 100

print("Username:", username)
print("Bio:", bio)
print("Followers:", followers)
```

- Run the code and observe: It is used to store information about the Instagram profile so that the values can be used and displayed later in the program.
 - What is the use of the variables "username", "bio" and "followers"?
 - What if you change these values, does the output change? Yes.
- You can check the difference in the repository files by typing:


```
git diff
```
- Add and commit the changes to your repository. The message in the commit helps you later understand what were the changes made to the files.

Activity 3: Follower Growth tracker

- Variables act as containers that store data and changing the value of a variable changes the output. Append the following code to your python file:

```
followers = 100

followers += 50
print("Day 1:", followers)

followers += 20
print("Day 2:", followers)

followers -= 10
print("Day 3:", followers)
```

- Run and observe:
 - Do we manually have to reassign the follower value? No.
 - Does each operation affect the first value (100) or the existing value of followers? Each operation affects the existing value of followers.
 - What is the use of += or -= operators? It adds/subtracts a value from the current value of the variable.
- Commit the changes to your repository

Activity 4: Interactive profile creator

1. Currently your program displays hard coded values. Let's try to make it more interactive and personalized:

```
username = input("Enter Username: ")
age = input("Enter Age: ")
category = input("Enter Content Category: ")

print("\nInstagram Profile")
print("=====")
print("Username:", username)
print("Age:", age)
print("Category:", category)
```

2. Run the code and observe:
 - a. How does input() captures user's input Prompts the user to enter their input and stores what they input in a variable.
 - b. Is the program dynamic now or still hard coded? Dynamic.
 - c. Try to run multiple types with different inputs
3. Commit the change to your repository

Activity 5: Something fun to think about. More on this next week:

1. Try this code:

```
username = input("Enter Username: ")
age = int(input("Enter Age: "))
category = input("Enter Content Category: ")

print("\nInstagram Profile")
print("=====")
print("Username:", username)
print("Age:", age)
print("Category:", category)

if age>40 and category == "fun":
    print("You are old what is fun for you??")
```

2. The output:

```
Enter Username: Nisha
Enter Age: 41
Enter Content Category: fun

Instagram Profile
=====
Username: Nisha
Age: 41
Category: fun
You are old what is fun for you??
```

3. What do you observe, apart from the fact that I am old:
 - a. What is the return type for input() It is string by default.
 - b. How are different conditions being checked? Different conditions are checked using comparison and logical operators.
4. Commit changes to your repository

What you need to submit?

1. Your python file (with all the 5 activities)
2. Your git log. Store it in a text file.

git log >log.txt

Your log file will have commits to every activity. It will look something like this:

```
commit 2a1c21990aadf492991d1ff32f65fa135cdc8941
Author: Nisha <nisha.jain@singaporetech.edu.sg>
Date: Thu Aug 20 11:46:00 2026 +0800

    Fun conditions

commit ec13a0d14059b57fdab27635554b27a2f05ebfce
Author: Nisha <nisha.jain@singaporetech.edu.sg>
Date: Thu Aug 20 11:34:43 2026 +0800

    Dynamic Profile

commit 2a1adcb852ae04a6f9163b8212d7ccfb4691dec5
Author: Nisha <nisha.jain@singaporetech.edu.sg>
Date: Thu Aug 20 11:33:10 2026 +0800

    Assignment Variables

commit 18139a65d339f104604784a686ba5982510089dc
Author: Nisha <nisha.jain@singaporetech.edu.sg>
Date: Thu Aug 20 11:31:08 2026 +0800

    Added Profile Variables

commit 14584ed1c8970359900feb7d1e2a0d823a34229b
Author: Nisha <nisha.jain@singaporetech.edu.sg>
Date: Thu Aug 20 10:47:26 2026 +0800

    First Post
```

3. Put the python file and log file in a folder. Zip it and upload to xSite